Project 09 – PowerShell-Based Windows System Inspection

Author: Madhuri

Date: 12 November 2026

Environment: Windows 10 Pro (64-bit) **Tool:** Windows PowerShell

1. Objective

To use PowerShell commands to inspect running Windows processes, examine Windows services, and collect operating system and computer configuration information.

The practical focuses on basic system inspection techniques used in system administration, troubleshooting, and cybersecurity investigations.

2. Environment and Tools

- Operating System: Windows 10 Pro
- OS Version: 10.0.19045
- OS Build: 19045
- System Architecture: 64-bit
- Command-Line Environment: Windows PowerShell

Device-specific identifiers and hardware details have been excluded for privacy.

3. Process Inspection

3.1 Display Running Processes

Command:

```
Get-Process
```

Output Observed: The command displayed running processes with properties such as process name, process ID, CPU time, and memory usage.

Examples observed included:

- chrome
- explorer
- powershell
- svchost
- MsMpEng
- postgres
- wps

Meaning: `Get-Process` retrieves information about currently running processes on Windows.

Security Relevance:

- Inspect active applications and background processes.
- Identify unfamiliar processes for further investigation.
- Review process IDs and resource usage.

- Support basic process investigation.

3.2 Inspect Chrome Processes

Command:

```
Get-Process -Name chrome
```

Output Observed: Multiple Chrome process entries were displayed, each with its own process ID and resource information.

Meaning: The command filters the running process list and displays processes matching the name `chrome`.

Security Relevance:

- Verify whether a specific application is running.
- Inspect multiple processes associated with an application.
- Support application and process investigation.

Observation: Multiple Chrome processes can be normal because Chrome uses a multi-process architecture. Their presence alone does not indicate malicious activity.

3.3 Sort Processes by CPU Time

Command:

```
Get-Process | Sort-Object CPU -Descending
```

Output Observed: Processes were displayed in descending order according to their cumulative CPU time.

Some processes appearing near the top included:

- Rave
- promecefpluginhost
- wps
- chrome
- msedgewebview2
- explorer

Meaning: `Sort-Object` sorts process objects using the CPU property. `-Descending` arranges the results from highest to lowest.

Security Relevance:

- Identify processes with higher cumulative CPU time.
- Investigate unusual resource consumption.
- Support performance troubleshooting and process analysis.

Observation: CPU time represents accumulated processor time, not current CPU usage percentage. Higher CPU time alone does not establish that a process is malicious.

4. Windows Service Inspection

4.1 Display Windows Services

Command:

```
Get-Service
```

Output Observed: The command displayed Windows services with their service names, display names, and current statuses. Both Running and Stopped service states were observed.

Examples included:

- Windows Event Log
- Windows Defender Firewall
- Microsoft Defender Antivirus Service
- Print Spooler
- Windows Management Instrumentation
- WLAN AutoConfig

Meaning: `Get-Service` retrieves registered Windows services and their current states.

Security Relevance:

- Monitor important Windows services.
- Review security-related service status.
- Identify unexpected service activity for further investigation.
- Support system administration and troubleshooting.

4.2 Inspect Print Spooler Service

Command:

```
Get-Service -Name Spooler
```

Output Observed: Name: Spooler | Display Name: Print Spooler | Status: Running

Meaning: The command retrieves the current status of the Print Spooler service, which manages printing operations.

Security Relevance: Demonstrates how to inspect an individual Windows service and verify its current status.

4.3 Filter Running Services

Command:

```
Get-Service | Where-Object {$_.Status -eq "Running"}
```

Output Observed: The command displayed services whose status was Running.

Security-related services observed included:

- BFE – Base Filtering Engine
- mpssvc – Windows Defender Firewall
- WinDefend – Microsoft Defender Antivirus Service
- WdNisSvc – Microsoft Defender Antivirus Network Inspection Service
- wscsvc – Security Center

Meaning: `Where-Object` filters service objects using a specified condition. This command displays only services with the status Running.

Security Relevance:

- Review active security-related services.
- Verify the current state of important services.
- Support basic security monitoring.

Observation: A stopped service is not automatically suspicious. Its status must be evaluated according to its purpose and configuration.

5. System Information Inspection

5.1 Collect Detailed Computer Information

Command:

```
Get-ComputerInfo
```

Output Observed: The command returned detailed operating system, BIOS, and computer configuration information.

- Windows Product Name: Windows 10 Pro
- Windows Version: 2009
- OS Version: 10.0.19045
- OS Build Number: 19045
- OS Architecture: 64-bit
- Domain: WORKGROUP
- Domain Role: Standalone Workstation
- Total Physical Memory: Approximately 8 GB

Meaning: `Get-ComputerInfo` collects detailed information about the operating system, BIOS, and computer configuration.

Security Relevance:

- Identify the operating system edition and build.
- Review system configuration.
- Support system inventory and security audits.
- Help assess operating system update requirements.

5.2 Retrieve Operating System Information

Command:

```
Get-CimInstance Win32_OperatingSystem
```

Output Observed: System Directory: C:\WINDOWS\system32 | Build Number: 19045 | Version: 10.0.19045

Meaning: The command retrieves operating system information through the Windows CIM infrastructure.

Security Relevance:

- Identify the installed operating system version and build.
- Support system inventory.
- Help assess operating system patch requirements.

5.3 Retrieve Computer System Information

Command:

```
Get-CimInstance Win32_ComputerSystem
```

Output Observed: Domain: WORKGROUP | Total Physical Memory: 8,482,041,856 bytes (approximately 7.9 GiB)

Meaning: The command retrieves computer system information, including manufacturer, model, domain membership, and physical memory. Device-identifying fields are intentionally excluded from this report.

Security Relevance:

- Collect hardware inventory information.
- Identify whether a computer belongs to a workgroup or domain.
- Support system audits and configuration reviews.

6. PowerShell Features Used

Feature	Purpose
`Get-Process`	Retrieves process information
`Get-Service`	Retrieves Windows service information
`Get-ComputerInfo`	Collects computer configuration
`Get-CimInstance`	Retrieves system information through CIM
`Where-Object`	Filters objects using conditions
`Sort-Object`	Sorts objects by property values
Pipeline (|)	Passes command output to another command

7. Security Observations

• Multiple Chrome processes were observed. This can be normal application behaviour and is not proof of malicious activity.

• Processes such as MsMpEng appeared in the process listing, corresponding to Microsoft Defender components.

• The Print Spooler service was observed in the Running state.

• Windows Defender Firewall and Microsoft Defender-related services appeared in the running-services output.

• Both running and stopped Windows services were observed. Their status must be evaluated according to their intended configuration.

• Process CPU time was reviewed using sorting. Cumulative CPU time should not be confused with current CPU utilization.

• Operating system and hardware configuration information was collected for basic system inventory and security assessment.

Investigation Limitation: This practical was limited to basic process, service, and system information inspection. It did not perform malware detection, process integrity verification, digital signature validation, or a complete security audit. No suspicious activity was confirmed by these commands alone.

8. Conclusion

This practical demonstrated the use of Windows PowerShell for inspecting running processes, monitoring Windows services, and collecting operating system and computer configuration information.

The exercise provided practical experience with process inspection, service monitoring, object filtering, sorting, pipelines, and CIM-based system information retrieval.

These techniques form a foundation for Windows system administration and can support further cybersecurity investigation and troubleshooting.

9. Evidence

Screenshots were captured during the practical to document command execution and observed outputs.

The evidence covers:

• Running process inspection

• Chrome process inspection

• CPU-time-based process sorting

• Windows service listing

• Print Spooler service status

• Running service filtering

- Detailed computer information
- Operating system information
- Computer system information

Privacy Note: Device names, usernames, SIDs, Product IDs, serial numbers, email addresses, IP addresses, and personal file paths have been excluded or should be redacted from publicly shared evidence.